

Review

Not peer-reviewed version

A Practical Tutorial on Agentic AI

Vinod Kumar Chauhan^{*}, Xin Wang, Jaspreet Singh, Yvonne Lu, Jiandong Zhou, Ruchit Agrawal

Posted Date: 25 August 2026

doi: 10.20944/preprints202608.1756.v1

Keywords: agentic AI; AI agents; large language models; multi-agent systems; artificial intelligence



Preprints.org is a free multidisciplinary platform providing preprint service that is dedicated to making early versions of research outputs permanently available and citable. Preprints posted at Preprints.org appear in Web of Science, Crossref, Google Scholar, Scilit, Europe PMC, OpenAlex.

Copyright: This open access article is published under a [Creative Commons CC BY 4.0 license](https://creativecommons.org/licenses/by/4.0/), which permit the free download, distribution, and reuse, provided that the author and preprint are cited in any reuse.

Disclaimer/Publisher's Note: The statements, opinions, and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions, or products referred to in the content.

Review

A Practical Tutorial on Agentic AI

Vinod Kumar Chauhan ^{1,*}, Xin Wang ², Jaspreet Singh ³, Yvonne Lu ², Jiandong Zhou ^{4,5,6} and Ruchit Agrawal ⁷

¹ Department of Computer and Information Sciences, University of Strathclyde, Glasgow, UK
² Department of Engineering Science, University of Oxford, Oxford, UK
³ School of Computing, Engineering and Digital Technologies, Teesside University, London, UK
⁴ Department of Family Medicine and Primary Care, The University of Hong Kong, Hong Kong, China
⁵ School of Public Health, The University of Hong Kong, Hong Kong, China
⁶ Department of Pharmacology and Pharmacy, The University of Hong Kong, Hong Kong, China
⁷ Department of Information Systems, Carnegie Mellon University, Doha, Qatar
* Correspondence: vinod.chauhan@strath.ac.uk

Abstract

Agentic AI is the most recent AI paradigm that extends the capabilities of large language models (LLMs) by providing access to external tools for performing specific tasks, external information resources for grounding responses rather than relying solely on model knowledge, and persistent task state and memory. These capabilities allow AI systems to plan, act, and adapt to intermediate results, whilst maintaining progress and coordinating specialised agents, allowing complex tasks to be completed with varying degrees of autonomy. For example, an agentic systematic-review assistant can search the web for relevant papers, screen and analyse them, and synthesise the evidence into a PRISMA-informed review. Research and development in agentic AI have expanded rapidly, accompanied by a growing range of general-purpose and specialised agent systems and development frameworks. Although several recent reviews have examined frameworks, evaluation, taxonomies and applications, practical hands-on guidance for researchers and practitioners seeking to understand and build agentic systems remains limited. This paper addresses this gap by providing a practical, implementation-oriented tutorial for building agentic AI systems using plain Python as well as three representative frameworks: LangGraph, CrewAI and the OpenAI Agents SDK. We discuss the progression from basic LLMs to agentic AI, explain the core components of agentic systems and examine agentic workflows and control patterns through which these components are combined to solve tasks; the tutorial then addresses evaluation at the component, trajectory and task levels, together with robustness, failure recovery and practical design guidance. Finally, we implement a research assistant as a case study to illustrate how agentic AI systems can be built, evaluated and safeguarded, supported by concise code skeletons in the paper and complete runnable implementations and additional end-to-end case studies in the accompanying repository at <https://github.com/jmdvinodjmd/agentic-ai-tutorial>.

Keywords: agentic AI; AI agents; large language models; multi-agent systems; artificial intelligence

1. Introduction

A conventional large language model (LLM) operates through a request–response pattern involving a single model invocation: a user sends a request, called a prompt, to an LLM, and the model-generated output is returned as a response [1,2]. This single-call request–response pattern works well for tasks that can be completed using the information available in the model’s parameters and the request context, such as summarising supplied text, translating or classifying a passage, and answering self-contained questions. However, many real-world tasks require systems to access up-to-date external information rather than rely only on the model’s parametric knowledge, maintain task state and progress across multiple steps, use task-specific tools to obtain information or perform actions, choose among possible actions, and adapt their subsequent actions based on intermediate

observations [3]. These requirements are particularly important in changing and time-sensitive settings such as healthcare monitoring, cybersecurity operations and disaster response, where applications must interpret new information and support timely action within defined human and operational controls [4,5].

Agentic AI addresses these requirements by extending LLM-based applications beyond response generation to support the execution of complex tasks. By combining language models with capabilities such as access to task-specific tools and external resources, retained task state, planning, feedback and coordination among specialised agents, these systems can undertake complex tasks over one or more steps and interact with external environments [6–8]. Agentic AI therefore concerns the design of the wider application rather than the language model alone: the model helps select actions, while application logic maintains state, executes validated actions and enforces operational constraints. Existing applications include software-engineering agents such as Codex, which can inspect codebases, modify code and iteratively run tests [9,10], and research agents that search multiple sources and synthesise cited reports [3]. Figure 1 provides a simplified view of this wider application.

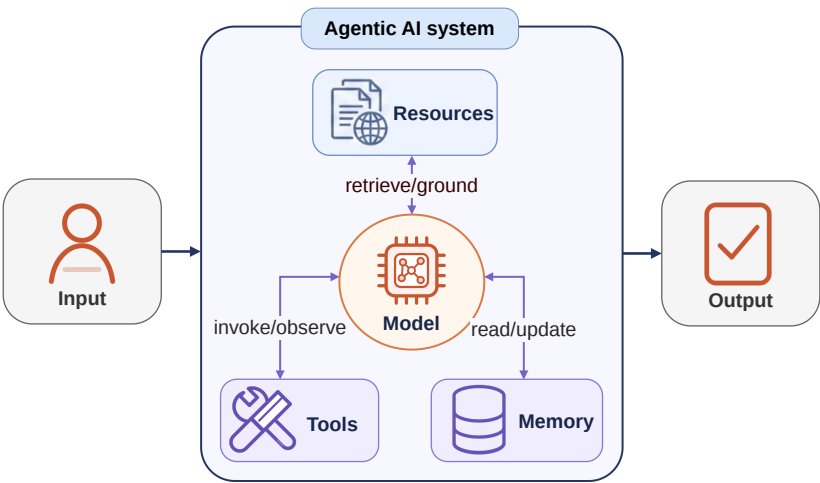


Figure 1. High-level view of an agentic AI application in which an LLM uses external resources, tools and memory to complete complex tasks, while application-defined logic controls these interactions and the overall execution.

Research and development in agentic AI have expanded rapidly, producing diverse architectural patterns, development frameworks and applications, while raising new evaluation and safety challenges. Several broad surveys examine agent architectures, components, tool use, single- and multi-agent systems, applications and open challenges [4–6,8,11–14]. More focused reviews examine planning [15], memory [16], framework architectures and communication protocols [17], and agent evaluation and benchmarking [18,19]. Domain- and context-specific works examine agentic programming and software development [20], scientific discovery [21], healthcare, including radiology and elderly care [22,23], and the adoption of interconnected agent systems in small, medium and micro-enterprises [24]. Practical guidance has also begun to emerge, e.g., Sypherd and Belle [25] organise implementation considerations around planning, memory, tools and control flow, with attention to stochasticity and resource use, and Bandara et al. [26] present a production-oriented approach to designing, developing and deploying agentic AI workflows through a multimodal case study. Among these, the work of Bandara et al. [26] is closest in scope to our tutorial, but its emphasis on production-grade engineering and best practices differs from our beginner-oriented, framework-independent approach. Our tutorial complements existing conceptual, empirical and practical work by connecting agentic AI concepts with implementation and evaluation.

This paper discusses the progression from basic LLM applications to agentic AI, the main components of agentic systems, and the workflow and control patterns through which these components are combined. It illustrates their implementation in plain Python and three representative frameworks: LangGraph, CrewAI and the OpenAI Agents SDK. The paper also covers evaluation at the component,

trajectory and task levels, together with robustness, failure recovery, safety, human oversight and practical guidance on using agentic approaches, before bringing these elements together through an integrated research-assistant case study supported by complete runnable implementations and additional end-to-end case studies in the accompanying repository.

The tutorial is structured as follows: Section 2 explains the progression from LLM applications to agentic AI. Sections 3 and 4 introduce the core system components and common execution patterns, respectively, before Section 5 presents their implementation using plain Python and representative frameworks. Sections 6 and 7 address evaluation, safety and practical guidance. Section 8 integrates the material through a research-assistant case study, and Section 9 provides concluding remarks.

2. From LLMs to Agentic AI

The progression from conventional LLM applications to agentic AI, as illustrated in Figure 2, can be understood through five key developments in application capability: single LLM calls, retrieval augmentation, tool use, stateful agentic workflows and multi-agent coordination. Across this progression, applications gain greater access to external information, a wider range of actions, retained task state, adaptive control and coordination among specialised agents. These developments became prominent at different points in recent LLM research, although their boundaries overlap and contemporary systems combine them according to the needs of the task [6,8,11]. The contemporary use of the term agent builds on established research in intelligent agents and multi-agent systems, where agents were commonly characterised by autonomy, reactivity, proactivity and social ability [27,28]. LLM-based agents extend this operational view by using language models to interpret task context, select actions and interact with tools, data sources, humans or other agents [11]. Current terminology remains unsettled, particularly over whether agentic AI requires multiple agents [8,12]. Throughout this tutorial, an AI agent denotes a model-centred operational entity that participates in selecting actions, and an agentic workflow denotes the execution structure combining model decisions with tools, state and application-level control. Agentic AI is used as the broader application-level term and may involve one or more agents operating over one or more steps; a multi-agent system specifically coordinates several interacting agents.

The progression towards agentic AI begins with a single LLM call, in which the model generates an output using the prompt and the information encoded in its parameters [1,2,29]. Retrieval-augmented generation (RAG) extended this arrangement by retrieving external information and adding it to the model context before generation [30]. The original RAG formulation established this approach in 2020, while later methods introduced more adaptive forms, including active retrieval to determine when and what to retrieve during generation and Self-RAG to combine retrieval with reflection over retrieved passages and generated responses [31,32]. Active retrieval and Self-RAG moved retrieval from a single preparatory operation towards a process that can respond to the evolving task.

The next major development connected LLMs with task-specific tools that can retrieve information, execute code, query databases or interact with software services [11]. Modular Reasoning, Knowledge and Language (MRKL) [33] introduced a modular architecture for routing requests to external reasoning and knowledge modules, while Toolformer [34] and Reasoning and Acting (ReAct) [35] demonstrated model-directed tool use and the integration of tool results into subsequent reasoning. In this arrangement, the model generates structured action requests whose execution remains controlled by application code, extending its role from producing an answer to influencing the application's behaviour through an action–observation cycle.

Stateful agentic workflows extend this cycle across a sequence of decisions and actions, with the application retaining the current task state, recording intermediate results and using new observations to determine subsequent steps. Its control flow may include routing, iteration, planning, critique, replanning and recovery, while application logic supplies the surrounding execution structure by managing state, validating actions and defining permissions, checkpoints, budgets and stopping conditions. Methods such as ReAct and Reflexion illustrate how observations and feedback can guide

later decisions, while stateful tool-use benchmarks show the importance of tracking intermediate progress across longer interactions [35–37].

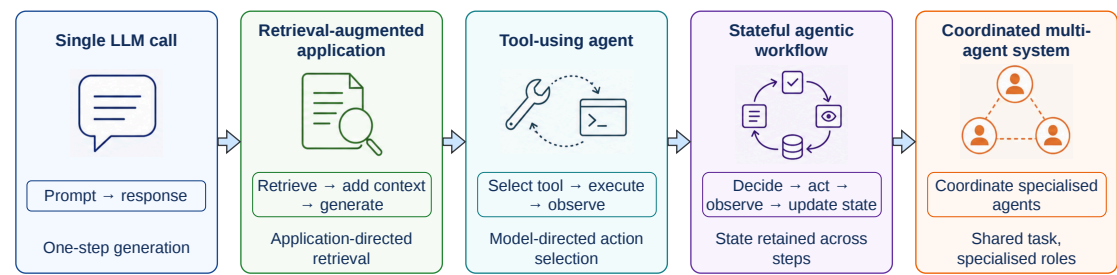


Figure 2. Progression from a single LLM call to agentic AI through retrieval augmentation, model-directed tool use, stateful execution and multi-agent coordination, with each stage introducing additional application capabilities.

A further development distributes different responsibilities across multiple interacting agents, drawing on established research on intelligent agents and multi-agent systems that is now extended through the reasoning and language capabilities of LLMs [28]. Each agent may have specialised instructions, tools, information or permissions, while coordination may occur through dialogue, delegation, shared state or a supervising agent. LLM-based systems such as CAMEL [38], AgentVerse [39] and MetaGPT [40] demonstrate role-based interaction, staged collaboration and structured coordination for reasoning and software-development tasks. Multi-agent organisation is particularly useful when a task contains separable responsibilities that benefit from specialisation, parallel work or independent review. Its design must also account for communication, coordination and the integration of outputs from different agents.

3. Components of an Agentic AI System

Drawing on the main elements identified in the literature [11,35], this section organises the components needed for an agentic system into five groups according to the responsibilities they fulfil: defining the task and the model interface; enabling action and information retention through tools, the environment and state; organising execution through planning and orchestration; checking decisions and results through validation and human oversight; and maintaining reliability through operational controls that keep execution bounded and support recovery. Listing 1 provides a minimal framework-independent execution skeleton in plain Python, illustrating the basic cycle that these component groups configure and extend.

The listing assumes that the task and model interface have already been defined: `initialise_state(task)` creates runtime state for the specified task, while `build_context()` and `request_decision()` prepare the model context and obtain a decision. `validate_decision()` checks the proposed action before `execute_action()` interacts with the tools and environment, and the resulting observation is incorporated into the state by `update_state()`. The loop provides the simplest form of orchestration; `validate_result()` checks the final result, `should_stop()` bounds execution and `checkpoint()` persists the current state. Explicit planning, human intervention, tracing, execution budgets and recovery are not represented directly in this minimal skeleton and are introduced in the corresponding subsections.

Listing 1. Minimal code skeleton of an agentic AI system in plain Python.

```
1 state = initialise_state(task)
2
3 while not should_stop(state):
4     context = build_context(state)
5     decision = request_decision(model, context)
6
7     action = validate_decision(decision, state)
```

```

8     observation = execute_action(action)
9
10    state = update_state(state, action, observation)
11    state = checkpoint(state)
12
13    return validate_result(state)

```

3.1. Task Specification and Model Interface

Before an application can select or execute an action, it needs a clear account of the task and a defined way of obtaining decisions from the model. Task specification and the model interface therefore establish what the application is expected to achieve and how model decisions enter the execution process.

Task and goal specification. A user request expresses a desired outcome, often without specifying all the operational details needed to pursue it, and a task specification translates this request into an operational definition of the work to be performed. It records the objective, required output, available actions, applicable constraints and criteria for successful completion [41], and may also define prohibited actions, resource limits and conditions for termination or escalation [42,43]. These elements should be represented in application state or executable policy where they can be checked during execution. The resulting specification gives the other components a common reference for deciding which actions are relevant, permitted and sufficient to complete the task.

Models, instructions and structured outputs. Once the task has been specified, the application calls a model at decision points that require interpretation, reasoning or selection among possible actions. System instructions describe the model's role, available actions, expected behaviour and task-specific constraints, giving the model the information needed to propose a suitable decision, while application code enforces permissions and operational limits when that proposal is processed. Structured outputs allow the application to use model decisions in a predictable form, and a schema may specify the selected action, its arguments, the reported status and the proposed next step [44,45]. The application can reject outputs that do not conform to this schema before they affect the execution. Schema compliance only establishes that the output has the expected structure, so a second validation step checks whether the proposed action is relevant to the current task, uses permissible values and complies with policy [37], even when the model output is syntactically valid.

3.2. Tools, Environment and State

A model can propose what should happen next, while tools allow the application to obtain information or carry out the proposed action and state records what happens during this interaction; together, these components connect model decisions to external resources and maintain the information needed as execution proceeds.

Tools and the external environment. A tool is an application interface through which an agent obtains information, performs a computation or acts on an external system [11]. For example, a research assistant may use one tool to search for papers, another to retrieve their contents and another to store selected results. Each tool should have a clear contract describing its purpose, input and output schemas, possible errors and side effects. These descriptions help the model choose an appropriate tool and supply the required arguments [46], after which application code validates the arguments and performs the call. Tools may be read-only or effectful, with search, retrieval and database queries commonly treated as read-only because they obtain information without changing the external system. Sending a message, modifying a record or executing a transaction changes an external system and therefore requires stronger permissions, confirmation and recovery support [37,47]. Effectful operations should be idempotent where possible, so that accidental repetition does not create an additional effect, while operations that cannot be made idempotent may require a compensating action. The environment contains the systems and resources upon which tools operate. After a tool runs, its result must be made available to the rest of the application by converting it into an observation that is

recorded in runtime state and includes the outcome of the action, relevant state changes, errors and provenance¹. This information allows the model to respond to the result in a subsequent decision and supports later audit or recovery [35].

State, memory and context management. As an application interprets a task, calls tools and receives observations, information accumulates and later components need a consistent account of it to avoid repeating completed work or acting on an outdated view of the task. An explicit runtime-state schema provides this account and allows the components of an agentic application to exchange the information consistently. Runtime state records the current execution, including completed steps, intermediate outputs, observations, pending actions, errors and remaining budgets, thereby providing the application with an authoritative account of progress. For example, the state can tell the planner which steps remain, the validator which action is being proposed and the controller whether a resource limit has been reached. The context window is the bounded amount of information that a model can process in one call. *Context* is the information selected for that call, such as instructions, observations, retrieved evidence and relevant parts of the runtime state. Because the full state may become too large or contain details that are irrelevant to the current decision, context management selects, orders and, when necessary, compresses the information supplied to the model [48]. Compression must preserve details needed for subsequent decisions. *Memory* stores information for reuse beyond the immediate execution, such as a user preference or knowledge obtained during an earlier task [16,49]. *Checkpoints* store durable snapshots from which a particular execution can resume. Memory should have a defined source, scope, retention period and update policy because stale or untrusted entries can influence future runs. Checkpoints require separate handling because they may contain pending actions and sensitive execution data.

3.3. Planning and Orchestration

Some objectives can be completed through one action, while others contain several activities that must occur in a particular order. Planning provides a route through these activities, while orchestration uses the plan and current state to determine what runs next.

Planning and task decomposition. Planning is useful when a task requires several dependent activities or when the route to completion may need to change during execution. For example, a research-assistant task may require forming a search query, retrieving papers, screening their relevance and synthesising the retained evidence, with task decomposition making these intermediate activities explicit and allowing their progress to be tracked [41]. A plan may be represented as a sequence, a dependency graph or another task structure suited to the application, with sequential plans being appropriate when each step follows the previous one and dependency graphs when some activities can run independently while others require particular results. Other planning methods may search among candidate routes or apply formal constraints where the task permits such a representation [50,51]. The selected representation should identify the intermediate tasks, their dependencies and the conditions under which they are ready to run. The application tracks which steps are pending, ready, completed, failed or no longer relevant, checks dependencies before execution and revises the plan when observations or failures invalidate the current route. Replanning requires a clear trigger and a revision limit to prevent repeated replacement of one incomplete plan with another.

Orchestration and control. A plan describes the intended route, while orchestration manages its execution by determining which component runs next and how information and control pass between components. Its responsibilities may include sequential execution, conditional routing, parallel work, retries, handoffs and coordination among specialised agents or workers [52–54]. Control decisions may be deterministic, model-directed or hybrid. Application code handles decisions that follow known rules, such as stopping when all acceptance criteria have been met. A model may select among permitted alternatives when the route depends on the meaning of the current task or observation, as

¹ *Provenance* records the origin and processing history of information, allowing an output to be traced through tool calls and intermediate transformations to its supporting sources.

in choosing a suitable tool after examining a search result [35]. In a hybrid design, application code defines the available transitions and the model makes bounded choices within them [43]. Keeping this control structure explicit supports testing, tracing and framework-independent reasoning about execution.

3.4. Validation and Human Oversight

Model decisions and tool results can be incomplete, incorrect or unsuitable for the current task. The application therefore needs ways to assess progress before accepting a result or carrying out a consequential action, with validation providing automated checks and human oversight supplying judgement and authority where needed.

Reflection, critique and external validation. Reflection examines an output or execution trajectory and may propose a revision when an initial result can be improved by identifying missing evidence, inconsistencies or unmet requirements. Reflexion [36] and Self-Refine [55] show that iterative feedback can improve performance on some tasks. Model-generated critique should be treated as a candidate judgement because a model may repeat the original error or incorrectly revise a valid result [56], so independent evidence is needed to establish whether an output is correct. This evidence may come from executable tests, retrieved evidence, external observations or the acceptance criteria defined in the task specification. Deterministic validators are appropriate for properties that can be specified mechanically, whereas model-based critics can assess properties that require semantic judgement if their conclusions are recorded for further checking. Revision limits and stopping conditions keep reflection bounded because repeated critique does not guarantee convergence.

Human-in-the-loop interaction. Some tasks involve missing information, ambiguous choices or actions whose consequences require human authority, so human oversight gives the application a defined way to obtain that information or authority instead of making an unrestricted decision during system design, execution or post-run review. At design time, people define goals, policies, permissions and acceptable risk [57]; during execution, the application may request missing information, ask for approval of a proposed action or escalate a decision that exceeds its authority [52,54,58]; and post-run review supports quality assurance, audit and system improvement. Runtime intervention should produce an explicit result such as approval, rejection, revision or a request for additional information, and any approval should apply to the exact action and arguments presented for review. Suitable intervention points include ambiguous decisions, irreversible effects and actions with material consequences, whereas routine low-risk operations can proceed within the permissions established during design.

3.5. Reliability and Operational Control

Because agentic tasks may span several model calls, tools and external services, an interruption or failed action can leave the application with incomplete work or an uncertain external state, making reliability and operational controls necessary to keep execution recoverable, bounded and observable.

Checkpointing, interruption and resumption. Checkpointing persists enough runtime state to pause and resume an execution, without which a long-running task may have to restart after a service failure, application interruption or request for human approval. Checkpointing is therefore useful for tasks that span multiple sessions or contain deliberate pauses [52,59]. A checkpoint may contain completed actions, pending decisions, observations, current budgets and the position of the execution controller. Safe resumption requires the application to determine whether earlier effectful actions were completed, with tool-call identifiers and recorded outcomes allowing it to continue from the last consistent state without repeating completed operations [37]. Checkpoints also support audit and recovery and should be protected according to the sensitivity of their contents.

Error recovery and stopping conditions. An application that interacts with models and external services must expect failures arising from invalid model outputs, tool errors, unavailable services, inconsistent state or changes in the external environment. Recovery options include bounded retries, alternative tools, fallback models, replanning, human escalation and resumption from a checkpoint

[41,43], with retries suited to failures expected to be transient. Before repeating an effectful action, the application requires an idempotency check or a compensating action. The application also needs to recognise when execution should finish, so every execution path requires explicit success, failure and budget-based stopping conditions. Success is determined from the task’s acceptance criteria. Failure conditions may include unrecoverable errors, repeated actions, lack of progress and policy violations [42,56]. When a time, iteration, token, tool-call or monetary limit is reached, the application terminates safely or escalates with its partial results and execution trace.

Guardrails, tracing and execution budgets. As an application gains access to tools and external systems, a faulty or manipulated model decision can have consequences beyond an incorrect textual response, so guardrails check model inputs, outputs, tool calls and observations before these results influence later steps or external systems. Prompt-based guidance and model classifiers can identify possible problems. Deterministic validation, least-privilege permissions, credential isolation and secure tool design enforce application boundaries [60–62]. Section 7 examines these controls and their limitations, including risks arising from prompt injection and untrusted tool inputs [63,64]. Tracing records observable events across an execution, including model requests, structured decisions, tool calls, observations, state transitions, approvals, errors and resource use, which are needed to reconstruct what happened and identify the source of a failure when several components contribute to the final result. These traces support debugging, provenance, evaluation, replay and audit. Traces should focus on observable actions and state changes rather than private chain-of-thought, which may be unavailable or unfaithful. Execution budgets limit cost and operational exposure by covering tokens, elapsed time, model calls, tool calls, monetary cost, iterations and effectful actions. The application should enforce these limits during execution and include their consumption in the trace. Together, explicit guardrails, traces and budgets keep execution bounded and inspectable.

4. Agentic Workflows and Control Patterns

This section explains how an application connects the components of an agentic AI system, discussed in the previous section, and controls their execution. A useful starting question is what determines the next step: in some tasks, the application follows a known sequence; in others, the next step depends on an observation obtained during execution; and a task may also be divided among specialised agents or workers. Throughout the section, we use a research assistant as a running example: given a research question, the assistant searches for relevant sources, extracts and assesses evidence, and produces a supported synthesis. The same components can be arranged in different ways depending on whether these activities follow a known route, adapt to intermediate results or require specialised participants.

The patterns are organised into three groups according to how they control execution: workflow-composition patterns arrange predefined stages and branches, iterative single-agent patterns use observations or evaluations to select subsequent actions, and multi-agent patterns distribute responsibilities among coordinated participants. An application may combine patterns from these groups according to the task structure, uncertainty and risk [43]. Simple requests can also bypass unnecessary planning, iteration or delegation and proceed directly to an appropriate tool or response. Figure 3 shows the characteristic control structure of each pattern, while Table 1 summarises when each pattern is useful and the main control required for its implementation.

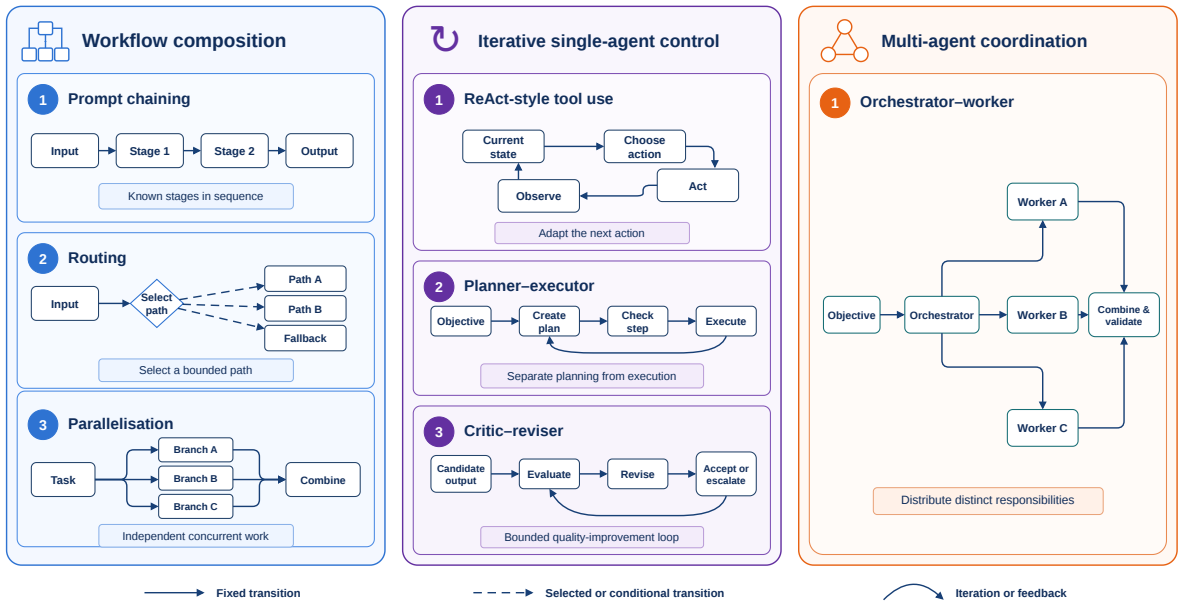


Figure 3. Common workflow and control patterns used in agentic systems, with each panel presenting the characteristic control structure of one pattern.

Table 1. Comparison of common workflow and control patterns used in agentic systems.

Pattern	Characteristic control	Most appropriate when	Principal control requirement
Prompt chaining	Predetermined sequence of model or software stages.	The task has stable stages and clear dependencies.	Validate intermediate outputs and define how the workflow handles an invalid or unexpected result.
Routing	Selection among enumerated models, tools, workflows or specialists.	Inputs require different capabilities or processing paths.	Constrain the available routes and provide a fallback for invalid or uncertain selections.
Parallelisation	Concurrent independent branches followed by aggregation.	Subtasks or alternative attempts are genuinely independent.	Define how results are combined and control duplicated work, inconsistency and cost.
ReAct-style tool use	Model-directed action-observation loop.	The next action depends on information discovered during execution.	Validate tool calls, bound the loop and treat observations according to their source and trust level.
Planner-executor	Validated plan followed by monitored execution and possible replanning.	A multi-stage task has identifiable subtasks and dependencies.	Check plan feasibility and dependencies, and trigger replanning only when conditions change.
Critic-reviser	Bounded evaluation and revision loop.	Outputs can be assessed against evidence, tests or explicit criteria.	Ground the critique, define acceptance criteria and limit the number of revisions.
Orchestrator-worker	Delegation to specialised workers followed by integration.	A task requires distinct tools, instructions, expertise or permissions.	Define worker responsibilities and permissions, and validate the integrated result.

4.1. Workflow Composition

Many tasks have identifiable stages even when one or more stages use a language model, and workflow-composition patterns encode this known structure in the application. The application determines the available stages and transitions, while models may make bounded decisions within individual stages.

Prompt chaining. Prompt chaining is useful when a task contains a stable sequence of activities and each activity depends on the result of an earlier one, with the application passing the output of one stage to the next and performing any required transformation or validation between them. For example, the research assistant may extract claims from retrieved documents, validate their supporting evidence and then pass the retained claims to a synthesis stage. Similar staged processes are used for decomposition and evidence-supported revision [41,65]. The application fixes the order of the stages and defines the condition for moving from one to the next [43], making intermediate validation important because an incorrect extraction can affect every later stage that consumes it. A failed validation can stop the chain, request human review or direct execution to a repair step. Tasks whose required sequence changes frequently are better served by routing or iterative control.

Routing. Routing is useful when different inputs require different capabilities or processing paths, with a router examining the request and selecting an appropriate model, tool, workflow or specialist from a defined set. In the research assistant, a literature query may be sent to a search workflow, while a request to analyse extracted numerical data may be sent to a data-analysis workflow. Modular systems such as MRKL use this form of selection to connect language models with external modules [33]. The set of valid routes and the information required by each route should be defined by the application. Deterministic routing applies known rules, such as selecting a workflow from a document type. Model-directed routing is useful when selection depends on the meaning of the request or current state [43]. The application should check that the selected route exists and is permitted, while an uncertain selection can trigger clarification, use a default workflow or escalate to a human.

Parallelisation. Parallelisation is useful when several activities can proceed independently, as it runs these activities concurrently and combines their outputs after completion. A research assistant may search several bibliographic databases at the same time or ask several evaluators to assess the same synthesis against different criteria. Parallel execution can reduce elapsed time and increase coverage [43]. The design must specify which activities are genuinely independent and how their outputs will be aggregated. Search results may require deduplication and ranking, while evaluator results may require a rule for resolving disagreement. Dependencies between branches should be identified before they are launched because parallel branches can otherwise repeat the same work, produce incompatible outputs or increase model and tool costs. Multi-agent systems may also use parallel workers when their assignments can be executed independently [38,39].

4.2. Iterative Single-Agent Control

Some tasks reveal the appropriate route only after execution has begun: a search result may suggest a better query, a tool may return an error or an intermediate output may fail its acceptance criteria. Iterative patterns return this new information to the decision process and use it to select or revise subsequent actions. ReAct supplies an action–observation loop, planner-executor maintains a task-level plan and critic-reviser adds explicit quality assessment.

ReAct-style tool use. ReAct-style tool use is appropriate when the next action depends on information obtained from the current one: the model selects an action, the application executes it and the resulting observation becomes part of the context for the next decision [35]. A research agent may formulate a search query, inspect the returned results, refine the query and retrieve a selected source before producing an answer. Tool calls and observations should be stored as structured operational events, allowing the application to update its state, validate the next action and reconstruct the execution later. The implementation does not require storing or exposing private chain-of-thought. An action–observation loop requires explicit operational controls: tool calls should be validated before execution, observations should be handled according to their source and trust level, and iteration, time and tool-use budgets should define how long the loop may continue. Exact-action approval is appropriate before an action with material external effects. AgentBench identifies a gap between language capability and reliable performance in interactive tasks [66]. Indirect prompt injection creates an additional risk because untrusted content returned by a tool can enter the observation used for the next decision [64].

Planner-executor. The planner-executor pattern is useful when a task contains several dependent stages whose overall structure can be identified before or during execution. The planner decomposes the objective and proposes an ordered task structure, while the executor performs the steps that are ready, records their outcomes and reports changes that affect the remaining plan. An executor may use a ReAct-style loop to complete an individual step, so the planner-executor pattern adds task-level organisation around action-level execution. A proposed plan must be checked before it controls execution, with the application verifying that its actions are available, its dependencies are consistent and its constraints match the task specification. This produces a validated executable plan from the model's proposal; the plan also provides a visible account of progress and can be reviewed before costly or effectful steps begin. Planning may produce a textual sequence, search among alternative routes or translate the task into a formal planning representation. Plan-and-Solve uses explicit task decomposition to support multi-step reasoning [41], while Tree of Thoughts explores alternative reasoning paths [51]. Formal planning can provide stronger validity or optimality guarantees when actions and constraints can be represented precisely [50]. Planning adds model calls and delays before action begins and is most useful when the dependencies and coordination benefits justify this overhead. Replanning should follow evidence that an action failed, an assumption changed or a required dependency is no longer available. A revision limit prevents execution from being displaced by repeated speculative planning.

Critic-reviser. The critic-reviser pattern is useful when a candidate output can be assessed against evidence, tests or explicit criteria: a critic examines the output and identifies specific deficiencies, and a reviser then addresses those deficiencies and returns the revised result for acceptance or another bounded evaluation. The pattern is also called evaluator–optimiser when evaluation and improvement are implemented as separate roles or components. ReAct uses observations to determine the next action, while critic-reviser uses an explicit assessment to improve a candidate result. The pattern can operate after a fixed workflow or within an iterative agent. In the research assistant, a critic may identify claims that lack support in the retrieved sources, after which the reviser removes, qualifies or supports them. Similar loops can check code against tests or a report against a required schema. Iterative feedback has improved outputs in Reflexion, Self-Refine and retrieval-supported revision settings [36,55,65]. Reliable evaluation requires a clear basis for judgement, with suitable foundations including retrieved evidence, deterministic tests, an independent evaluator and the acceptance criteria defined for the task. Human review can supply expert judgement when these criteria cannot be applied automatically. Because a model may repeat its original error when asked to evaluate unsupported reasoning or revise a correct response incorrectly [56], the application should record the evidence behind each criticism, define the condition for accepting the result and limit the number of revision rounds. An unresolved result can be returned with its limitations or escalated for review.

4.3. Multi-Agent Coordination: Orchestrator–Worker

A task may contain responsibilities that require different instructions, tools, expertise or permissions, which multi-agent coordination assigns to specialised participants while defining how their work contributes to the shared objective. This tutorial focuses on the orchestrator-worker pattern because it provides a clear and controlled introduction to multi-agent execution. In the orchestrator-worker pattern, an orchestrator decomposes the task, assigns subtasks to suitable workers and integrates their outputs, with related terms including supervisor–specialist, manager–agent and agents-as-tools. Implementations range from model-backed functions that share one application state to more independent agents with their own state and tools. For the research assistant, one worker may search for sources, another may extract claims and a third may assess methodological quality, with the orchestrator supplying each worker with a structured assignment that defines its objective, inputs, available tools and expected output. Each worker should receive the information and permissions required for that assignment. An effectful or high-impact delegated action remains subject to the application's approval policy.

Role-based communication and specialised collaboration have been applied to cooperative reasoning [38,39], software development [40] and long-form evidence synthesis [67], with these systems varying in how much control remains with the orchestrator, how state is shared and how freely workers communicate. Delegation creates a need for explicit integration because workers may duplicate work, use inconsistent assumptions or return outputs that cannot be combined directly. The orchestrator should track responsibility for each subtask, validate worker outputs and resolve conflicts before producing the final result. Clear assignments and scoped permissions also control token use and limit the effects of an incorrect worker decision. The patterns in this section differ in how the next step is selected, how intermediate results influence execution and whether responsibility is concentrated in one workflow or distributed among workers, but they can be composed within the same application. The research assistant may route a request to the appropriate workflow, search independent sources in parallel, use a planner to organise the review, assign evidence gathering to ReAct-style workers and apply a critic-reviser loop to the synthesis.

Each additional branch, loop or worker adds execution and evaluation work, so the application should use the simplest arrangement that meets the task requirements. A request that can be answered through one validated retrieval step should proceed directly to that step, with planning, iteration and delegation introduced only where the task provides a specific reason for them.

5. Agentic AI Implementations

This section discusses how an agentic AI system is implemented in executable software using plain Python as well as in representative agentic development frameworks. Plain Python uses data structures, functions, conditions and loops for this purpose, while specialised frameworks provide reusable abstractions for recurring requirements such as model invocation, tool registration, state management, routing, persistence, human intervention and tracing. This section compares a plain Python baseline with three frameworks: LangGraph represents execution as a stateful graph [52], CrewAI organises work through agents, tasks, crews and flows [53], and the OpenAI Agents SDK provides an agent runner together with tools and delegation mechanisms [54]. These approaches place the responsibilities introduced earlier in different parts of the implementation.

All four examples address the same bounded research task of retrieving evidence for a question, extracting and validating relevant claims and synthesising a cited answer. To support a matched comparison, they use the same task input, model configuration, tool contracts, structured schemas, acceptance criteria, stopping conditions and execution budgets. Only their control structures differ, allowing each example to show the characteristic abstraction of its implementation approach and the comparison to examine how the same application responsibilities are represented in plain Python and the three frameworks. The listings are implementation skeletons that show the code needed to understand each control abstraction while omitting provider configuration, supporting function definitions and other setup. Complete runnable versions and tests are provided in the accompanying repository.

5.1. Plain Python Baseline

In the plain Python baseline, state is stored in an application-defined data structure, tools are exposed as validated functions and execution is controlled through ordinary conditions, loops and function calls. Listing 2 follows the execution sequence of the shared research task. The application first creates a `ResearchState` containing the user's question and the available budget, which records progress, observations and accumulated evidence. The loop continues until `should_stop()` detects successful completion, failure or exhaustion of a budget. Within each iteration, `model_decide()` requests a proposed action from the model and `validate_decision()` checks that proposal against the current task and permissions. A sensitive action is passed to `request_approval()`, an application-defined function that returns an executable decision after approval and handles rejection by stopping or redirecting the execution. The approved action is then performed by `tools.execute()`, which returns an observation. The application incorporates the decision and observation through `update_state()`

and records the operational event through `trace.record()`. Once execution stops, `synthesise()` produces an answer from the accepted evidence and `validate_answer()` checks the result against the task's acceptance criteria.

Listing 2. Plain Python implementation skeleton.

```

1 state = ResearchState(question=question, budget=budget)
2
3 while not should_stop(state):
4     decision = validate_decision(model_decide(state))
5
6     if requires_approval(decision):
7         decision = request_approval(decision)
8
9     observation = tools.execute(decision)
10    state = update_state(state, decision, observation)
11    trace.record(state, decision, observation)
12
13 answer = validate_answer(synthesise(state.evidence))

```

Plain Python is suitable for small or stable workflows and for applications requiring close integration with existing software, while also providing full control over data structures and execution behaviour. The developer implements persistence, interruption, tracing, retries and concurrency according to the application's requirements, but as the number of branches and recovery paths grows, these responsibilities occupy more of the application code and become more difficult to inspect as a complete control structure.

5.2. LangGraph

LangGraph is an open-source orchestration framework within the LangChain ecosystem for building long-running, stateful agentic applications [52]. It represents a workflow with several branches, cycles and recovery paths as a directed graph in which nodes perform operations and edges determine which operation runs next. The graph representation maps the responsibilities visible in the plain Python baseline to explicit framework elements. Application functions become nodes, the runtime state becomes a shared graph schema, conditions become routing functions and conditional edges, and cyclic edges represent iteration. This makes the possible execution paths visible in the graph definition. Listing 3 applies this representation to the research task. StateGraph creates a graph whose nodes share the application-defined ResearchState, while calls to `add_node()` associate the names `retrieve`, `extract_claims`, `validate_claims` and `synthesise` with the functions that perform those operations. `START` and `END` mark the boundaries of execution, and `add_edge()` defines the fixed transitions between stages. Validation can produce more than one permitted next step, and the `add_conditional_edges()` call associates `route_after_validation()` with two possible outcomes: a retry returns execution to retrieval, while accept proceeds to synthesis. The routing function therefore occupies the same conceptual position as a conditional statement in the plain Python implementation. The call to `compile()` creates the executable application, with the supplied `checkpoint` persisting graph state at execution boundaries [52]. Finally, `invoke()` starts the graph from `initial_state` using the supplied runtime config and returns a result containing the final state and output.

Listing 3. LangGraph implementation skeleton.

```

1 graph = StateGraph(ResearchState)
2
3 graph.add_node("retrieve", retrieve)
4 graph.add_node("extract", extract_claims)
5 graph.add_node("validate", validate_claims)
6 graph.add_node("synthesise", synthesise)

```

```

7
8 graph.add_edge(START, "retrieve")
9 graph.add_edge("retrieve", "extract")
10 graph.add_edge("extract", "validate")
11 graph.add_conditional_edges(
12     "validate",
13     route_after_validation,
14     {
15         "retry": "retrieve",
16         "accept": "synthesise",
17     },
18 )
19 graph.add_edge("synthesise", END)
20
21 app = graph.compile(checkpointer=checkpointer)
22 result = app.invoke(initial_state, config=config)

```

Checkpointing allows graph execution to pause and resume from persisted state, and LangGraph interrupts use this capability to wait for external input, including human approval, before execution continues [52]. In the research workflow, an interrupt could pause execution before accessing a restricted source or publishing a synthesis, after which the resumed graph can continue from its saved position without repeating completed nodes. Nodes that create external effects should still use idempotency checks because retries and resumption may revisit an interrupted operation. LangGraph is suited to applications whose execution contains stateful branching, cycles, interruption points or several recovery routes. A clear state schema determines what information is available across nodes. Node boundaries should correspond to meaningful operations, and routing functions should return a defined set of transitions. Representing every minor operation as a node produces a graph that is difficult to read, so closely related operations can remain within one node when they share the same control boundary.

5.3. CrewAI

Some tasks are easiest to describe through the responsibilities assigned to different participants, and CrewAI is an open-source framework for constructing such role-oriented and multi-agent applications [53]. It provides agents, tasks, crews, processes and flows at different levels of the implementation. An *agent* represents a role with a goal, instructions and available tools. A *task* defines a unit of work, its expected output and the agent responsible for completing it. A *crew* groups agents and tasks, while a *process* determines how the tasks are executed. CrewAI supports sequential and hierarchical processes [53]. A *flow* can provide broader application-level state, event handling and routing around crews and other application operations. Listing 4 demonstrates a sequential crew, while CrewAI flows are discussed separately below as a broader application-control abstraction. The listing uses three agents to represent the specialised responsibilities of the research task. Agent creates each specialist from a role, an operational goal, contextual backstory and any callable tools. The application-defined `search_tool()` and `retrieve_tool()` allow the researcher to search for and retrieve evidence, while `verification_tool()` allows the critic to validate extracted claims. The writer receives no external tool in this skeleton because its assigned task is to synthesise the validated evidence. Each Task defines a description, an expected_output and the agent assigned to perform the work, while the context field passes the output of an earlier task to a later one. The critic therefore receives the output of `research_task`, and the writer receives the output of `critique_task`. Crew combines the agents and tasks, while `Process.sequential` executes the tasks in their declared order. Calling `kickoff()` starts the crew and returns its final result [53].

Listing 4. CrewAI implementation skeleton.

```

1 researcher = Agent(

```

```

2     role="Evidence researcher",
3     goal="Retrieve and extract relevant evidence",
4     backstory="A specialist in literature retrieval",
5     tools=[search_tool, retrieve_tool],
6 )
7
8 critic = Agent(
9     role="Evidence critic",
10    goal="Validate claims against retrieved evidence",
11    backstory="A specialist in evidence verification",
12    tools=[verification_tool],
13 )
14
15 writer = Agent(
16     role="Synthesis writer",
17     goal="Produce a concise, cited answer",
18     backstory="A specialist in evidence synthesis",
19 )
20
21 research_task = Task(
22     description=question,
23     expected_output="Relevant claims with source references",
24     agent=researcher,
25 )
26
27 critique_task = Task(
28     description="Validate the extracted claims",
29     expected_output="Accepted claims with validation results",
30     agent=critic,
31     context=[research_task],
32 )
33
34 writing_task = Task(
35     description="Produce a cited answer",
36     expected_output="A cited answer based on accepted claims",
37     agent=writer,
38     context=[critique_task],
39 )
40
41 result = Crew(
42     agents=[researcher, critic, writer],
43     tasks=[research_task, critique_task, writing_task],
44     process=Process.sequential,
45 ).kickoff()

```

This representation makes the division of work visible through named roles and tasks and is suitable when the responsibilities require different instructions, tools, expertise or permissions. The same task may need fewer agents when its stages can be implemented reliably as deterministic functions or performed by one tool-using agent. Reliable coordination requires explicit interfaces between the roles. The task description defines what each agent should do, and the expected-output field describes what the next task should receive. Structured output schemas and validation at task boundaries provide stronger interfaces than role descriptions alone. A CrewAI flow can add application-controlled routing, persistent state and human intervention when the execution extends beyond the sequential process shown in the listing [53].

5.4. OpenAI Agents SDK

The OpenAI Agents SDK is a lightweight framework for constructing tool-using and multi-agent applications [54], with its main abstractions comprising agents, tools, a runner and delegation: an *agent* combines a model with instructions, tools and an optional structured output type, while the *runner*

executes the agent loop until it obtains a final output or reaches an execution limit. Specialist agents can participate through two delegation mechanisms: an agent can be exposed as a tool, allowing the current agent to call the specialist and retain responsibility for the overall task, while a handoff transfers control to the receiving agent. The appropriate mechanism depends on which agent should remain responsible for subsequent decisions and the final output [54]. Listing 5 uses agents as tools. Agent creates the researcher, critic and writer from their names, application-defined instructions, available tools and structured output_type. The schemas EvidenceBundle, ValidatedEvidence and CitedAnswer define the outputs expected from each role. The calls to as_tool() expose the researcher and critic as callable tools, with the tool_name providing the programmatic identifier presented to the writer and the tool_description explaining when the specialist should be used. The writer acts as the central orchestrator because both specialist tools remain available within its agent loop. Runner.run_sync() starts the writer with the supplied question, and the runner processes model outputs and tool calls until the writer produces a CitedAnswer or execution reaches a stopping condition [54], returning a result object containing the typed final output and the operational items produced during the run.

Listing 5. OpenAI Agents SDK implementation skeleton.

```

1  researcher = Agent(
2      name="Evidence researcher",
3      instructions=research_instructions,
4      tools=[search_tool, retrieve_tool],
5      output_type=EvidenceBundle,
6  )
7
8  critic = Agent(
9      name="Evidence critic",
10     instructions=critique_instructions,
11     tools=[verification_tool],
12     output_type=ValidatedEvidence,
13 )
14
15 research_tool = researcher.as_tool(
16     tool_name="research_evidence",
17     tool_description="Retrieve and extract relevant evidence",
18 )
19
20 critique_tool = critic.as_tool(
21     tool_name="validate_evidence",
22     tool_description="Validate claims against the evidence",
23 )
24
25 writer = Agent(
26     name="Synthesis writer",
27     instructions=writing_instructions,
28     tools=[research_tool, critique_tool],
29     output_type=CitedAnswer,
30 )
31
32 result = Runner.run_sync(writer, question)

```

The agents-as-tools arrangement gives the writer responsibility for integrating and validating the final answer. A handoff would be suitable if the receiving specialist should take control of the remaining interaction. This distinction allows delegation to follow the responsibility structure of the application. The SDK also provides sessions, structured outputs, guardrails, approval mechanisms and tracing [54]. Within this execution model, sessions can retain conversational history across related runs; input, output and tool guardrails apply checks at defined execution boundaries; application permissions and credential controls govern which external actions can occur; and tracing records

model calls, tool use, handoffs and other operational events. Applications requiring durable recovery across process failures or long interruptions need an appropriate persistence and resumption design in addition to the agent loop.

5.5. Matched Comparison

Table 2 compares how different implementation approaches represent the same application responsibilities through their respective abstractions. The selection of the implementation approach depends on the control structure required to solve a problem. Plain Python keeps each responsibility in application code and may be sufficient for small, stable control structures or highly customised integrations. LangGraph represents state transitions, branches and resumption through a graph, making it suitable for long-running tasks that require explicit execution paths and durable interruption. CrewAI represents responsibilities through roles and assigned tasks, suiting work with a meaningful division among specialists. The OpenAI Agents SDK provides a compact representation for model-directed loops that combine tool use with selective delegation through agents-as-tools or handoffs. Existing infrastructure, deployment requirements, observability and team experience should also inform the selection.

Table 2. Comparison of the principal agentic implementation approaches.

Dimension	Plain Python	LangGraph	CrewAI	OpenAI Agents SDK
Core abstraction	Functions, typed data and explicit control logic	Stateful directed graph	Agents, tasks, crews and flows	Agent loop, tools and delegation
State handling	Developer-defined typed state	Shared graph-state schema	Task context and flow state	Sessions and developer-managed application context
Orchestration	Conditions, loops and function calls	Nodes, edges and conditional routing	Crew processes and event-driven flows	Runner loop, tools and handoffs
Cycles and iteration	Explicit loops	Cyclic graph edges	Flow or task-level control	Runner loop and application logic
Multi-agent support	Developer-defined coordination	Graph composition and supervisor structures	Role-based crews	Handoffs and agents-as-tools
Human oversight	Application-defined approval logic	Graph interruption and resumption	Human feedback within flows	Tool approval and interruption mechanisms
Checkpointing and resumption	Application-defined persistence	Checkpoint-based graph persistence	Flow persistence where configured	Sessions; durable process recovery requires application infrastructure
Guardrails and validation	Application-defined checks	Validation within nodes and routes	Task and flow guardrails	Input, output and tool guardrails
Tracing	Application-defined event logging	Graph and execution tracing	Framework observability	SDK tracing
Implementation responsibility	State, control and operational services	State schema, nodes and routes	Roles, tasks, process and flow design	Agents, tools, delegation and application state
Most appropriate when	Control is small or highly customised	Execution requires stateful branching, cycles or durable resumption	Work maps to distinct specialist responsibilities	A compact tool-using loop and delegation model fits the task

5.6. Additional Frameworks

The ecosystem of agentic AI frameworks continues to expand rapidly, with alternatives such as Google’s Agent Development Kit [68] and Microsoft Agent Framework [59] providing further abstractions for agents, tools, state and workflow control. This tutorial focuses on LangGraph, CrewAI and the OpenAI Agents SDK because they represent distinct implementation approaches based on graph orchestration, role-oriented coordination and model-directed tool use and delegation, respectively; applying these frameworks alongside the plain Python baseline to the same bounded task, inputs, tool contracts, execution budgets and success criteria allows direct comparison of how they represent common application responsibilities. For broader coverage of agent frameworks, their architectures and communication protocols, please refer to the review by Derouiche et al. [17].

6. Evaluation of Agentic Systems

Evaluation begins by defining a representative task set that includes ordinary, difficult and failure-inducing cases, together with the operating environment, available actions, acceptance criteria, execution budget and versioned fixtures, prompts, schemas and tool responses required for repeated runs, and then assesses both execution and outcome at three nested levels: components, execution trajectories and task outcomes [19]. Figure 4 summarises this structure and the cross-cutting assessment of robustness, failure recovery, efficiency, human intervention and reproducibility used below for the research-assistant case study and the matched implementations from Section 5.

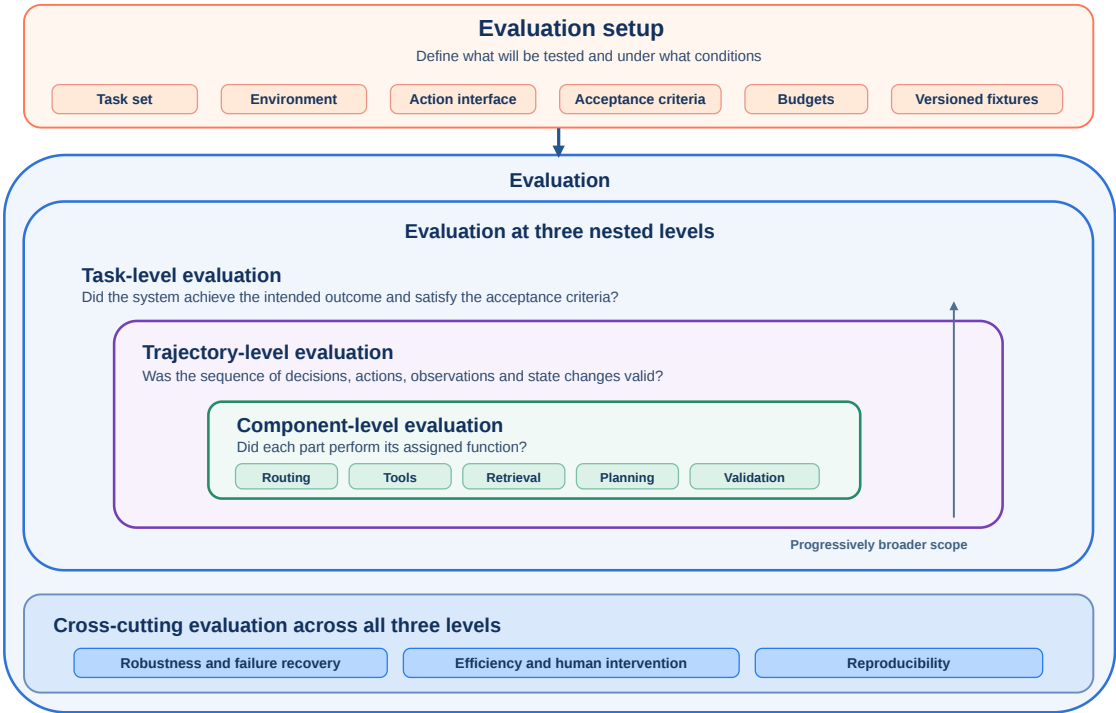


Figure 4. Evaluation structure for an agentic AI system: the setup defines the tasks, operating conditions and success criteria, after which evaluation proceeds across nested component, trajectory and task levels, while robustness, failure recovery, efficiency, human intervention and reproducibility are assessed across all three levels.

6.1. Component-Level Evaluation

Component-level evaluation tests individual operations before they are combined in an end-to-end execution, helping to locate failures in retrieval, routing, tool use, planning, memory, validation or another local function. Interface and control tests cover routing decisions, structured-output conformance, tool selection and argument validity; model-dependent tests assess retrieval, planning, memory and critique; and application-specific tests examine the operations required by the task. An empirical study of 39 open-source agent frameworks and 439 agentic applications found that more

than 70% of testing effort targeted deterministic resource and coordination components, fewer than 5% targeted model-based planning and approximately 1% examined prompts [69], indicating that testing must cover both conventional software behaviour and model-dependent decisions. Tool evaluation should distinguish among irrelevant selection, failure to select a required tool, malformed arguments and semantically incorrect arguments because selecting an appropriate tool is distinct from generating a valid call [46], just as schema conformance is distinct from the semantic correctness of the supplied arguments [45]. Deterministic or virtualised tools provide stable responses that allow failures to be attributed to the application rather than a changing external API [70]; planning tests can assess subtask coverage, ordering, dependency validity, feasibility and compliance with task constraints; and critics and validators should be tested with both correct and defective outputs because a critic that recommends unnecessary changes may reduce the quality of valid results, while unsupported self-correction does not consistently improve reasoning [56]. For the research assistant, component tests cover query routing, retrieval relevance, extraction of claims and source locations, construction of schema-valid evidence records, detection of unsupported claims and whether each citation supports its associated claim [71].

6.2. Trajectory-Level Evaluation

Trajectory-level evaluation examines how components interact during an execution through the decisions, actions, observations, state transitions, approvals, errors, recovery actions and termination event that connect the initial request to the final outcome [72]. It can identify repeated searches, unnecessary tool calls, invalid actions, inconsistent state, policy violations and incorrect stopping behaviour, while intermediate-progress evaluation distinguishes immediate failure from a run that reaches a later stage before stalling [73]. A canonical operational trace should record each event's type, timestamp, state identifier, invoked model or tool, validated arguments, observation, resulting state, approval decision, resource use and any associated error or recovery action so that task milestones and invariants can be checked. In the research assistant, these invariants require source retrieval to precede claim extraction, provenance to be attached before evidence is accepted and claim validation to precede synthesis, while execution should remain incomplete when a required approval is unresolved and should not repeat an identical failed query indefinitely. Schema validity, permitted tool use, action ordering and budget compliance can be assessed deterministically, whereas questions such as whether another search was useful or whether a synthesis accurately represents a source may require expert or model-based assessment; when a model judge is used, its prompt, model version, criteria and outputs should be recorded so that its judgements can be examined alongside the operational trace [19]. Private reasoning traces are unsuitable as evaluation evidence because they may be unavailable or may not faithfully represent the basis of a decision.

6.3. Task-Level Evaluation

Task-level evaluation measures whether the application achieves the outcome defined in its task specification using metrics appropriate to the task and environment. Exact or short-answer scoring may suit tasks with well-defined answers, as in GAIA [74]; software-engineering tasks can be assessed by applying generated changes and executing tests, as in SWE-bench [75]; and interactive web tasks can be evaluated through the resulting environment state, as in WebArena [47]. Offline evaluation compares an answer or predicted action with a fixed record, whereas interactive evaluation executes the action and observes its effect, which is necessary when later decisions depend on changes produced by earlier actions. Open-ended tasks require several outcome measures, and the research assistant should therefore be assessed for whether its answer addresses the question, covers the relevant evidence, represents sources accurately, handles contradictions and records sufficient provenance, with claim-level support and citation correctness assessed separately from fluency and presentation [76]. Component-level citation checks can be aggregated at this level and supplemented by citation completeness, which measures whether all claims requiring support have an appropriate citation [71].

6.4. Robustness, Failure Recovery and Reproducibility

Robustness testing changes selected parts of the evaluation setup and measures whether required behaviour is preserved by varying inputs through paraphrased requests, reordered information or ambiguous instructions, varying evidence through irrelevant material, missing sources or contradictions, and varying capabilities by removing a tool, changing its schema or altering the information it returns. Adversarial tests can place instruction-bearing content in retrieved documents or tool observations, as demonstrated by InjecAgent for indirect prompt injection through tool-integrated environments [64] and AgentDojo for the security evaluation of tool-using agents [61]. Recovery testing uses explicit fault injection, including tool timeouts, unavailable services, malformed model outputs, invalid arguments, partial state updates and interruption after an effectful action, to verify that the system detects the fault and selects an appropriate retry, fallback, replanning or escalation action. The resulting trace should show that completed work was preserved, duplicated external effects were avoided and unsafe recovery was escalated; because recovery remains less established than final task success in many agent benchmarks, these behaviours require transparent application-specific tests [37]. Repeated runs under the same versioned task, fixtures, model settings and budgets should report the distribution of outcomes, failure types, compliance and cost because agentic systems may follow different trajectories for the same input. Best-run performance estimates potential capability, for which $\text{pass}@k$ measures whether at least one of k attempts succeeds, whereas pass^k measures whether all k attempts succeed and therefore captures the consistency of acceptable behaviour [42]; policy-sensitive applications should additionally report how often every run both completes successfully and remains compliant. Reproducibility requires recording model and tool versions, prompts, schemas, sampling parameters, budgets, fixtures, environment versions and canonical traces, while virtualised environments can reduce variation caused by changing external services [70], although interactive environments may still change as their underlying applications and data evolve [47].

6.5. Efficiency and Human Intervention

Efficiency evaluation should report the computational resources used to obtain an outcome, including elapsed time, tokens, model calls, tool calls, iterations and monetary cost, with these values derived from the execution traces used for trajectory evaluation and compared under equivalent budgets. Planning, searching over alternatives and repeated revision add inference-time computation: Tree-of-Thoughts-style search evaluates several candidate reasoning paths [51], while iterative self-correction uses additional model calls whose contribution must be separated from that of the underlying method [56]. Cost-matched evaluation accounts for these differences and prevents improvements produced by additional computation from being attributed solely to the selected control pattern or framework. Human intervention should be evaluated both as a resource and as a control over execution through measures such as approval requests, requests for information, corrections, rejections, escalations, review time and residual error after intervention. The trace should record whether intervention occurred at the intended decision point and whether it prevented or corrected an error; for the research assistant, this may include reviewing selected sources, correcting extracted claims and approving the final synthesis. Comparisons between automatic and human-assisted execution should report any change in output quality together with the additional review burden because, although human feedback can improve planning and execution, intervention procedures and effort measures remain inconsistent across current evaluations [58].

7. Safety and Practical Guidance

An agentic application connects model decisions to tools, external information, stored state and other system components, creating practical questions about which decisions should involve a model, what authority each component should receive and how the application should respond when an action fails. Safety becomes especially important when an incorrect decision can disclose information, change an external system or affect later steps. Earlier sections introduced the mechanisms used to

manage these concerns, and this section briefly describes the main operational risks and consolidates the preceding guidance into a checklist for system design and review.

7.1. Principal Operational Risks

Unsafe actions and excessive permissions. A model may select an unsuitable tool, provide incorrect arguments or misjudge the effect of an action, with the consequences depending on the authority available to that tool. Applications should validate the proposed tool, arguments, task relevance, permissions and expected effect before execution, and tools that modify records, disclose information, send messages, execute code or initiate transactions require tightly scoped permissions and execution credentials. Generated code and other high-risk operations should run in isolated environments with restricted access to files, networks, processes and credentials. Agent-safety evaluations show that apparently reasonable model decisions can still produce harmful effects when these controls are absent [57,60].

Prompt injection and untrusted information. Because agents often process web pages, documents, emails, database records and tool outputs, they are exposed to indirect prompt injection when such content contains instructions intended to redirect the model, extract information or trigger an unauthorised action [61,63,64]. Applications should record the origin and trust level of external content, separate it from system instructions and check any resulting action against task policy and permissions. Filters may identify suspicious content; enforced permissions, isolated credentials and independent action validation limit the effects of content that reaches the model [77].

Contamination of state, memory and inter-agent communication. Incorrect, private or malicious information may persist in runtime state or memory and influence later decisions, while shared state and messages can carry this information from one agent to another. Memory entries should record their source, scope, creation time and trust status, with their management governed by policies for validation, access, correction, expiry and deletion. Sensitive information should be retained only for a defined application need, and information received from another agent should preserve its provenance [16,48].

Uncontrolled execution and accumulated effects. A multi-step workflow may repeat failed actions, continue without progress or create long chains of delegation, causing cost and risk to accumulate across the execution, particularly when tools modify external state. The stopping conditions, budgets and recovery mechanisms introduced earlier limit this exposure by detecting repeated actions, persistent failures, lack of progress and excessive delegation during execution. The execution record should identify completed effects so that recovery does not repeat them [42,62].

7.2. Practical Design Guidance

Practical design begins by defining the task requirements and possible consequences, which determine the appropriate architecture and degree of autonomy: model-directed decisions are useful when the next step depends on interpreting the current context, predefined program logic is appropriate when the required decision can be expressed as a rule, and consequential actions may require approval from a person with the relevant authority [43,57,58]. Table 3 consolidates these and other design principles introduced throughout the tutorial into a checklist for planning an application, selecting an implementation approach and reviewing a system throughout its specification, implementation, evaluation and deployment, helping to make important decisions explicit and keep the selected architecture proportionate to the task.

Table 3. Practical guidance for designing agentic AI systems.

Principle	Guidance
Task before framework	Define the objective, required output, permitted actions, constraints, acceptance criteria and consequences of failure before selecting a framework.
Minimum autonomy	Use model-directed control only where the task requires interpretation or adaptation. Implement known decisions and routine operations through predefined program logic.
Functional multi-agent design	Introduce multiple agents when the task contains genuinely distinct responsibilities. Define their authority, communication routes and delegation limits.
Explicit state	Represent completed steps, pending actions, observations, errors and remaining budgets in clear and preferably typed structures.
Governed memory	Define the source, scope, retention period and update policy of stored information, with procedures for validation, access, correction and deletion.
Trust separation	Distinguish system instructions from retrieved documents, messages and tool outputs, and preserve the provenance and trust level of external information.
Least privilege	Give each tool or worker only the permissions, data access and execution identity required for its assigned task. Keep credentials outside prompts and model-visible observations.
Decision–execution separation	Treat a model output as a proposed decision. Validate its relevance, arguments, permissions and expected effect before performing the action.
Deterministic validation	Use schemas, policy checks, executable tests and predefined rules for requirements that can be checked mechanically.
Exact-action approval	Present the proposed action, arguments, expected effect and supporting information for review. Bind approval to that specific proposal.
Runtime limits and circuit breakers	Enforce limits on iterations, time, tokens, tool calls, cost, delegation and effectful actions. Stop execution when it repeatedly fails or ceases to make progress.
Recovery by design	Use checkpoints, idempotent operations, compensating actions, safe defaults and explicit escalation paths for interrupted or partially completed work.
Operational tracing	Record model requests, structured decisions, tool calls, observations, approvals, state transitions, errors and resource use.
Process evaluation	Evaluate component behaviour, execution trajectories, policy compliance, recovery and repeated-run reliability together with final task outcomes.

8. Integrated Case Study

This section integrates the components, control patterns, evaluation methods and safeguards introduced earlier in a research assistant for evidence synthesis, a suitable case study because the task requires retrieval, tool use, state management, iterative control, validation, provenance and explicit stopping conditions. For the worked question, *Which interventions are effective at reducing household food waste, and under what conditions?*, the assistant searches for evidence, retrieves relevant passages, extracts and validates claims and produces an evidence-grounded synthesis, returning a supported answer, qualified answer, partial result or abstention according to the available evidence.

8.1. Task and Environment

The assistant operates over a bounded, versioned catalogue containing source identifiers, titles, passages, publication metadata and stable source locations, which supports repeatable success and failure cases under controlled conditions, although the same retrieval interface can also connect to a live scholarly search service; retrieved content is treated as untrusted data and cannot alter the application’s instructions, permissions or control flow. The environment exposes four principal operations: search accepts a structured query and returns ranked source records, retrieval returns passages and metadata

for selected sources, extraction converts relevant passages into typed claim proposals, and validation checks whether each claim is supported by its associated passage and linked to a valid source location.

The LLM handles semantic decisions such as query formulation, source selection, claim extraction, synthesis and critique, while application code executes the tools, validates structured outputs and enforces permissions, budgets and stopping conditions, with a generic LLM interface allowing the underlying model to be changed without altering the workflow. Runtime state records the question, search history, selected sources, retrieved passages, claim–evidence ledger, unresolved conflicts, draft answer, validation results, execution trace and remaining budgets; each ledger entry links a normalised claim to supporting, conflicting or unclear passages and preserves their source identifiers, locations and retrieval events, while the execution trace records searches, tool calls, state transitions, revisions and stopping decisions. Evidence coverage, validation results and remaining budgets determine when the task is complete.

8.2. Plain Python Implementation

The plain Python implementation expresses these responsibilities directly in application code rather than through framework-specific abstractions. Listing 6 accepts a generic question, searchable catalogue, configured llm interface and execution budget, with `initialise()` creating the runtime state and `build_context()` selecting the information needed for the next decision. Functions such as `generate_search_decision()` and `extract_claims()` construct task-specific instructions, invoke the configured LLM and parse its responses into typed structures, while model-generated search decisions, source selections and claims are treated as proposals whose corresponding validation functions determine whether they can proceed, leaving tool execution, state updates, checkpointing and termination under application control.

Listing 6. Plain Python research-assistant skeleton.

```

1  def run_research_assistant(
2      question, catalogue, llm, budget
3  ):
4      state = initialise(question, catalogue, budget)
5
6      while not evidence_ready(state):
7          context = build_context(state)
8          proposal = generate_search_decision(
9              llm, context
10             )
11          decision = validate_search_decision(
12              proposal, state
13             )
14
15          if decision.action == "clarify":
16              return request_information(decision.question)
17
18          results = search_catalogue(
19              catalogue, decision.query
20             )
21          state = record_search(
22              state, decision, results
23             )
24
25          proposed_sources = rank_sources(
26              llm, results, state
27             )
28          sources = validate_source_selection(
29              proposed_sources, results
30             )
31
32          for source in sources:
```

```

33     passage = retrieve_source(catalogue, source)
34     proposals = extract_claims(llm, passage)
35
36     for claim in proposals:
37         checked = validate_claim(claim, passage)
38         state = update_ledger(state, checked)
39
40     state = checkpoint(state)
41     if should_stop_searching(state):
42         break
43
44     draft = synthesise_answer(llm, state.ledger)
45     critique = critique_answer(
46         llm, draft, state.ledger
47     )
48     revised = revise_answer(llm, draft, critique)
49     answer = validate_answer(revised, state.ledger)
50
51     return finalise_or_abstain(answer, state)

```

The first phase constructs the evidence needed for the answer through a bounded ReAct-style search loop in which each result updates the state and informs the next proposed action; selected sources are then retrieved, their proposed claims are checked against the associated passages and accepted evidence is added to the ledger. Initial extraction from one source does not depend on another, so this stage can be parallelised in a complete implementation, although the minimal listing presents it as a sequential loop. The second phase constructs and checks the answer by synthesising the validated ledger, comparing the draft with the recorded evidence and acceptance criteria, revising identified problems and validating the resulting answer, with grounding in retrieved passages and typed claims making the critique available for external checking [56,65]. The checkpoint preserves progress after each search iteration, the supplied budget limits search, token, time, cost and revision use, and `finalise_or_abstain()` uses the validation results and recorded insufficiencies to select the appropriate outcome. The accompanying repository provides complete implementations of this workflow, including its prompts, schemas, tools, model adapters, fixtures and tests, in plain Python, LangGraph, CrewAI and the OpenAI Agents SDK; these implementations accept other research questions, and the repository also contains additional end-to-end case studies.

8.3. Evaluation and Safeguards

The case study applies the evaluation structure from Section 6: component-level tests cover search routing, retrieval relevance, source selection, output-schema validity, claim–passage support and citation validation; trajectory-level assertions check the ordering of search, retrieval, extraction, validation and synthesis together with state transitions and stopping decisions; and task-level evaluation examines question coverage, factual support, treatment of conflicting evidence, citation correctness, citation completeness and clarity about limitations. Identical versioned evidence fixtures and expected claim–evidence relationships ensure that the matched implementations are assessed using the same evidence and success conditions, allowing their trajectories and resource use to be compared.

Safeguard evaluation tests missing or conflicting evidence and verifies that the resulting limitations are reported, while malformed model outputs and unsupported claims must be rejected before entering the ledger. Tool-failure cases cover unavailable services and invalid responses [70], prompt-injection cases place untrusted instructions in retrieved content and verify that they cannot alter system policy [61,64], and interruption and budget-exhaustion cases check that completed actions remain recorded, recovery does not repeat their effects and the final outcome reflects the available evidence, with intermediate milestones and explicit state dependencies making these recovery behaviours observable [37].

9. Conclusion

Agentic AI has emerged as a new paradigm for developing AI systems that can pursue a goal over several steps and adjust their behaviour as a task unfolds, making it particularly useful when the next action depends on information discovered during execution, as in research assistance, web interaction and software development [6,8]. Its main advantage is this capacity to adapt: the model interprets new information and proposes how to proceed, and the surrounding application provides the tools and controls needed to carry out those decisions. The main challenge in developing agentic AI is maintaining reliable behaviour during task execution because, for example, an early error can influence later steps, self-correction can repeat the same mistake, and untrusted information obtained through tools can affect subsequent decisions [56,61,63,64]. Addressing these problems requires reliability to be designed across the full execution using measures such as action validation and operational limits, tracing and recovery, and evaluation at the component, trajectory and task levels. Human responsibility remains part of this process because people and organisations define acceptable risk, determine what the system is authorised to do and respond when it fails [57,58].

This tutorial has provided a beginner-oriented and implementation-focused account of agentic AI by explaining the progression from basic LLM applications to multi-step agentic systems, organising their main components and control patterns, and showing how the same application responsibilities can be represented in plain Python, LangGraph, CrewAI and the OpenAI Agents SDK. It has connected these implementation choices with multi-level evaluation, robustness and failure recovery, operational safeguards and human oversight, and brought them together in a research assistant for evidence synthesis, with complete matched implementations and additional end-to-end case studies provided in the accompanying repository. The tutorial is necessarily selective: its framework examples compare implementation abstractions rather than empirical performance, its evaluation framework provides a practical approach to testing rather than a large-scale empirical comparison, and specific implementation details will continue to change as models and frameworks evolve.

The central conclusion is that agentic AI is fundamentally a system-design problem: adaptive model behaviour becomes practically useful only when it operates within explicit application controls proportionate to the task and its possible consequences. Agentic systems should therefore use only the autonomy required by the task and keep model-directed decisions within application controls proportionate to their possible consequences.

Funding: This study received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Availability of Code: The code is available at <https://github.com/jmdvinodjmd/agentic-ai-tutorial>.

Use of Generative AI: Generative AI was used solely to assist with language editing and improving clarity. The authors critically reviewed and revised all content and take full responsibility for the final manuscript.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. Language Models are Few-Shot Learners. In Proceedings of the Advances in Neural Information Processing Systems; Larochelle, H.; Ranzato, M.; Hadsell, R.; Balcan, M.; Lin, H., Eds. Curran Associates, Inc., 2020, Vol. 33, pp. 1877–1901.
2. Ouyang, L.; Wu, J.; Jiang, X.; Almeida, D.; Wainwright, C.; Mishkin, P.; Zhang, C.; Agarwal, S.; Slama, K.; Ray, A.; et al. Training language models to follow instructions with human feedback. In Proceedings of the Advances in Neural Information Processing Systems; Koyejo, S.; Mohamed, S.; Agarwal, A.; Belgrave, D.; Cho, K.; Oh, A., Eds. Curran Associates, Inc., 2022, Vol. 35, pp. 27730–27744. <https://doi.org/10.52202/068431-2011>.
3. Taherinezhad, M.; Maier, S.; Vitagliano, G.; Pierri, F.; Feuerriegel, S. AutoSynthesis: An agentic system for automated meta-analysis, 2026, [arXiv:cs.AI/2607.15247].

4. Acharya, D.B.; Kuppan, K.; Divya, B. Agentic AI: Autonomous Intelligence for Complex Goals—A Comprehensive Survey. *IEEE Access* **2025**, *13*, 18912–18936. <https://doi.org/10.1109/ACCESS.2025.3532853>.
5. Brohi, S.; Mastoi, Q.u.a.; Jhanjhi, N.Z.; Pillai, T.R. A Research Landscape of Agentic AI and Large Language Models: Applications, Challenges and Future Directions. *Algorithms* **2025**, *18*. <https://doi.org/10.3390/a18080499>.
6. Wang, L.; Ma, C.; Feng, X.; Zhang, Z.; Yang, H.; Zhang, J.; Chen, Z.; Tang, J.; Chen, X.; Lin, Y.; et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science* **2024**, *18*, 186345.
7. Zhao, P.; Jin, Z.; Cheng, N. An In-depth Survey of Large Language Model-based Artificial Intelligence Agents, 2023, [\[arXiv:cs.CL/2309.14365\]](https://arxiv.org/abs/2309.14365).
8. Sapkota, R.; Roumeliotis, K.I.; Karkee, M. AI Agents vs. Agentic AI: A Conceptual taxonomy, applications and challenges. *Information Fusion* **2026**, *126*, 103599. <https://doi.org/10.1016/j.inffus.2025.103599>.
9. Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; de Oliveira Pinto, H.P.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. Evaluating Large Language Models Trained on Code, 2021, [\[arXiv:cs.LG/2107.03374\]](https://arxiv.org/abs/2107.03374).
10. OpenAI. Codex CLI documentation, 2026. Accessed 17 July 2026.
11. Chowda, S.S.; Alvi, R.; Rahman, S.S.; Rahman, M.A.; Raiaan, M.A.K.; Islam, M.R.; Hussain, M.; Azam, S. From language to action: a review of large language models as autonomous agents and tool users. *Artificial Intelligence Review* **2026**.
12. Bandi, A.; Kongari, B.; Naguru, R.; Pasnoor, S.; Vilipala, S.V. The rise of agentic ai: A review of definitions, frameworks, architectures, applications, evaluation metrics, and challenges. *Future Internet* **2025**, *17*, 404.
13. Khalid, O.; Farooqi, A.U.H.; Bilal, M. Agentic AI: a review, applications and open research challenges. *Computing* **2026**, *108*, 80.
14. Ferrag, M.A.; Tihanyi, N.; Debbah, M. From llm reasoning to autonomous ai agents: A comprehensive review. *IEEE Access* **2026**.
15. Huang, X.; Liu, W.; Chen, X.; Wang, X.; Wang, H.; Lian, D.; Wang, Y.; Tang, R.; Chen, E. Understanding the planning of LLM agents: A survey, 2024, [\[arXiv:cs.AI/2402.02716\]](https://arxiv.org/abs/2402.02716).
16. Zhang, Z.; Dai, Q.; Bo, X.; Ma, C.; Li, R.; Chen, X.; Zhu, J.; Dong, Z.; Wen, J.R. A Survey on the Memory Mechanism of Large Language Model-based Agents. *ACM Trans. Inf. Syst.* **2025**, *43*. <https://doi.org/10.1145/3748302>.
17. Derouiche, H.; Brahmi, Z.; Mazeni, H. Agentic ai frameworks: Architectures, protocols, and design challenges. *arXiv preprint arXiv:2508.10146* **2025**.
18. Mohammadi, M.; Li, Y.; Lo, J.; Yip, W. Evaluation and Benchmarking of LLM Agents: A Survey. In Proceedings of the Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2, New York, NY, USA, 2025; KDD '25, p. 6129–6139. <https://doi.org/10.1145/3711896.3736570>.
19. Yehudai, A.; Eden, L.; Li, A.; Uziel, G.; Zhao, Y.; Bar-Haim, R.; Cohan, A.; Shmueli-Scheuer, M. A Survey on Evaluation of LLM-based Agents. In Proceedings of the Findings of the Association for Computational Linguistics: ACL 2026; Liakata, M.; Moreira, V.P.; Zhang, J.; Jurgens, D., Eds., San Diego, California, United States, 2026; pp. 26690–26714. <https://doi.org/10.18653/v1/2026.findings-acl.1330>.
20. Wang, H.; Gong, J.; Zhang, H.; Xu, J.; Wang, Z. AI Agentic Programming: A Survey of Techniques, Challenges, and Opportunities, 2025, [\[arXiv:cs.SE/2508.11126\]](https://arxiv.org/abs/2508.11126).
21. Gridach, M.; Nanavati, J.; Abidine, K.Z.E.; Mendes, L.; Mack, C. Agentic AI for Scientific Discovery: A Survey of Progress, Challenges, and Future Directions, 2025, [\[arXiv:cs.CL/2503.08979\]](https://arxiv.org/abs/2503.08979).
22. Dietrich, N. Agentic AI in radiology: emerging potential and unresolved challenges. *British Journal of Radiology* **2025**, *98*, 1582–1584.
23. Khalil, R.A.; Ahmad, K.; Ali, H. Redefining Elderly Care With Agentic AI: Challenges and Opportunities. *IEEE Open Journal of the Computer Society* **2026**, *7*, 326–342. <https://doi.org/10.1109/OJCS.2026.3650842>.
24. Olujimi, P.A.; Owolawi, P.A.; Mogase, R.C.; Wyk, E.V. Agentic AI frameworks in SMMEs: a systematic literature review of ecosystemic interconnected agents. *AI* **2025**, *6*, 123.
25. Sypherd, C.; Belle, V. Practical Considerations for Agentic LLM Systems, 2024, [\[arXiv:cs.AI/2412.04093\]](https://arxiv.org/abs/2412.04093).
26. Bandara, E.; Gore, R.; Foytik, P.; Shetty, S.; Mukkamala, R.; Rahman, A.; Liang, X.; Bouk, S.H.; Hass, A.; Rajapakse, S.; et al. A practical guide for designing, developing, and deploying production-grade agentic ai workflows. *arXiv preprint arXiv:2512.08769* **2025**.
27. Wooldridge, M.; Jennings, N.R. Intelligent agents: theory and practice. *The Knowledge Engineering Review* **1995**, *10*, 115–152. <https://doi.org/10.1017/S0269888900008122>.
28. Botti, V. Agentic AI and Multiagentic: Are We Reinventing the Wheel?, 2025, [\[arXiv:cs.MA/2506.01463\]](https://arxiv.org/abs/2506.01463).

29. Rohanian, O.; Jauncey, H.; Nouriborji, M.; Kumar, V.; Goncalves, B.P.; Kartsonaki, C.; Merson, L.; Clifton, D.; Group, I.C.C.; et al. Using bottleneck adapters to identify cancer in clinical notes under low-resource constraints. In Proceedings of the Proceedings of the 22nd Workshop on Biomedical Natural Language Processing and BioNLP Shared Tasks, 2023, pp. 62–78.
30. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.t.; Rocktäschel, T.; et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* **2020**, *33*, 9459–9474.
31. Jiang, Z.; Xu, F.; Gao, L.; Sun, Z.; Liu, Q.; Dwivedi-Yu, J.; Yang, Y.; Callan, J.; Neubig, G. Active Retrieval Augmented Generation. In Proceedings of the Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing; Bouamor, H.; Pino, J.; Bali, K., Eds., Singapore, 2023; pp. 7969–7992. <https://doi.org/10.18653/v1/2023.emnlp-main.495>.
32. Asai, A.; Wu, Z.; Wang, Y.; Sil, A.; Hajishirzi, H. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In Proceedings of the International conference on learning representations, 2024, Vol. 2024, pp. 9112–9141.
33. Karpas, E.; Abend, O.; Belinkov, Y.; Lenz, B.; Lieber, O.; Ratner, N.; Shoham, Y.; Bata, H.; Levine, Y.; Leyton-Brown, K.; et al. MRKL Systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning, 2022, [\[arXiv:cs.CL/2205.00445\]](https://arxiv.org/abs/2205.00445).
34. Schick, T.; Dwivedi-Yu, J.; Dessi, R.; Raileanu, R.; Lomeli, M.; Zettlemoyer, L.; Cancedda, N.; Scialom, T. Toolformer: Language Models Can Teach Themselves to Use Tools, 2023, [\[arXiv:cs.CL/2302.04761\]](https://arxiv.org/abs/2302.04761).
35. Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.R.; Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models. In Proceedings of the The Eleventh International Conference on Learning Representations, 2023.
36. Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; Yao, S. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems* **2023**, *36*, 8634–8652.
37. Lu, J.; Holleis, T.; Zhang, Y.; Aumayer, B.; Nan, F.; Bai, H.; Ma, S.; Ma, S.; Li, M.; Yin, G.; et al. ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. In Proceedings of the Findings of the Association for Computational Linguistics: NAACL 2025; Chiruzzo, L.; Ritter, A.; Wang, L., Eds., Albuquerque, New Mexico, 2025; pp. 1160–1183. <https://doi.org/10.18653/v1/2025.findings-naacl.65>.
38. Li, G.; Hammoud, H.; Itani, H.; Khizbullin, D.; Ghanem, B. Camel: Communicative agents for" mind" exploration of large language model society. *Advances in neural information processing systems* **2023**, *36*, 51991–52008.
39. Chen, W.; Su, Y.; Zuo, J.; Yang, C.; Yuan, C.; Chan, C.M.; Yu, H.; Lu, Y.; Hung, Y.H.; Qian, C.; et al. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In Proceedings of the International Conference on Learning Representations, 2024, Vol. 2024, pp. 20094–20136.
40. Hong, S.; Zhuge, M.; Chen, J.; Zheng, X.; Cheng, Y.; Wang, J.; Zhang, C.; Yau, S.; Lin, Z.; Zhou, L.; et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In Proceedings of the International Conference on Learning Representations, 2024, Vol. 2024, pp. 23247–23275.
41. Wang, L.; Xu, W.; Lan, Y.; Hu, Z.; Lan, Y.; Lee, R.K.W.; Lim, E.P. Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models. In Proceedings of the Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers); Rogers, A.; Boyd-Graber, J.; Okazaki, N., Eds., Toronto, Canada, 2023; pp. 2609–2634. <https://doi.org/10.18653/v1/2023.acl-long.147>.
42. Yao, S.; Shinn, N.; Razavi, P.; Narasimhan, K.R. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In Proceedings of the The Thirteenth International Conference on Learning Representations, 2025.
43. Anthropic. Building effective agents, 2024. Accessed 17 July 2026.
44. Patil, S.G.; Zhang, T.; Wang, X.; Gonzalez, J.E. Gorilla: Large Language Model Connected with Massive APIs. In Proceedings of the The Thirty-eighth Annual Conference on Neural Information Processing Systems, 2024.
45. Patil, S.G.; Mao, H.; Yan, F.; Ji, C.C.J.; Suresh, V.; Stoica, I.; Gonzalez, J.E. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In Proceedings of the Forty-second International Conference on Machine Learning, 2025.

46. Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In Proceedings of the The Twelfth International Conference on Learning Representations, 2024.
47. Zhou, S.; Xu, F.F.; Zhu, H.; Zhou, X.; Lo, R.; Sridhar, A.; Cheng, X.; Ou, T.; Bisk, Y.; Fried, D.; et al. WebArena: A Realistic Web Environment for Building Autonomous Agents. In Proceedings of the The Twelfth International Conference on Learning Representations, 2024.
48. Packer, C.; Wooders, S.; Lin, K.; Fang, V.; Patil, S.G.; Stoica, I.; Gonzalez, J.E. MemGPT: Towards LLMs as Operating Systems, 2024, [arXiv:cs.AI/2310.08560].
49. Park, J.S.; O'Brien, J.; Cai, C.J.; Morris, M.R.; Liang, P.; Bernstein, M.S. Generative Agents: Interactive Simulacra of Human Behavior. In Proceedings of the Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, New York, NY, USA, 2023; UIST '23. <https://doi.org/10.1145/3586183.3606763>.
50. Liu, B.; Jiang, Y.; Zhang, X.; Liu, Q.; Zhang, S.; Biswas, J.; Stone, P. LLM+P: Empowering Large Language Models with Optimal Planning Proficiency, 2023, [arXiv:cs.AI/2304.11477].
51. Yao, S.; Yu, D.; Zhao, J.; Shafran, I.; Griffiths, T.; Cao, Y.; Narasimhan, K. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* **2023**, *36*, 11809–11822.
52. LangChain. LangGraph documentation, 2026. Accessed 17 July 2026.
53. CrewAI. CrewAI documentation, 2026. Accessed 17 July 2026.
54. OpenAI. OpenAI Agents SDK documentation, 2026. Accessed 17 July 2026.
55. Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; Prabhume, S.; Yang, Y.; et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems* **2023**, *36*, 46534–46594.
56. Huang, J.; Chen, X.; Mishra, S.; Zheng, H.S.; Yu, A.W.; Song, X.; Zhou, D. Large Language Models Cannot Self-Correct Reasoning Yet. In Proceedings of the The Twelfth International Conference on Learning Representations, 2024.
57. of Standards, N.I.; Technology. Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile. Technical Report NIST AI 600-1, National Institute of Standards and Technology, 2024. <https://doi.org/10.6028/NIST.AI.600-1>.
58. Pasukmit, J.; Takerngsaksiri, W.; Thongtanunam, P.; Tantithamthavorn, C.; Zhang, R.; Wang, S.; Jiang, F.; Li, J.; Cook, E.; Chen, K.; et al. Human-In-The-Loop Software Development Agents: Challenges and Future Directions. In Proceedings of the 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR), 2025, pp. 756–757. <https://doi.org/10.1109/MSR66628.2025.00112>.
59. Microsoft. Microsoft Agent Framework documentation, 2026. Accessed 17 July 2026.
60. Ruan, Y.; Dong, H.; Wang, A.; Pitis, S.; Zhou, Y.; Ba, J.; Dubois, Y.; Maddison, C.J.; Hashimoto, T. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In Proceedings of the The Twelfth International Conference on Learning Representations, 2024.
61. DeBenedetti, E.; Zhang, J.; Balunovic, M.; Beurer-Kellner, L.; Fischer, M.; Tramèr, F. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In Proceedings of the The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track, 2024.
62. OWASP Foundation. OWASP Top 10 for LLM applications 2025, 2025. Accessed 17 July 2026.
63. Greshake, K.; Abdelnabi, S.; Mishra, S.; Endres, C.; Holz, T.; Fritz, M. Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection, 2023, [arXiv:cs.CR/2302.12173].
64. Zhan, Q.; Liang, Z.; Ying, Z.; Kang, D. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In Proceedings of the Findings of the Association for Computational Linguistics: ACL 2024; Ku, L.W.; Martins, A.; Srikumar, V., Eds., Bangkok, Thailand, 2024; pp. 10471–10506. <https://doi.org/10.18653/v1/2024.findings-acl.624>.
65. Gao, L.; Dai, Z.; Pasupat, P.; Chen, A.; Chaganty, A.T.; Fan, Y.; Zhao, V.; Lao, N.; Lee, H.; Juan, D.C.; et al. RARR: Researching and Revising What Language Models Say, Using Language Models. In Proceedings of the Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers); Rogers, A.; Boyd-Graber, J.; Okazaki, N., Eds., Toronto, Canada, 2023; pp. 16477–16508. <https://doi.org/10.18653/v1/2023.acl-long.910>.
66. Liu, X.; Yu, H.; Zhang, H.; Xu, Y.; Lei, X.; Lai, H.; Gu, Y.; Ding, H.; Men, K.; Yang, K.; et al. AgentBench: Evaluating LLMs as Agents. In Proceedings of the The Twelfth International Conference on Learning Representations, 2024.

67. Shao, Y.; Jiang, Y.; Kanell, T.; Xu, P.; Khattab, O.; Lam, M. Assisting in Writing Wikipedia-like Articles From Scratch with Large Language Models. In Proceedings of the Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers); Duh, K.; Gomez, H.; Bethard, S., Eds., Mexico City, Mexico, 2024; pp. 6252–6278. <https://doi.org/10.18653/v1/2024.naacl-long.347>.
68. Google. Agent Development Kit documentation, 2026. Accessed 17 July 2026.
69. Hasan, M.M.; Li, H.; Fallahzadeh, E.; Rajbahadur, G.K.; Adams, B.; Hassan, A.E. An empirical study of testing practices in open source AI agent frameworks and agentic applications. *Empirical Software Engineering* **2026**, *31*, 124.
70. Guo, Z.; Cheng, S.; Wang, H.; Liang, S.; Qin, Y.; Li, P.; Liu, Z.; Sun, M.; Liu, Y. StableToolBench: Towards Stable Large-Scale Benchmarking on Tool Learning of Large Language Models. In Proceedings of the Findings of the Association for Computational Linguistics: ACL 2024; Ku, L.W.; Martins, A.; Srikumar, V., Eds., Bangkok, Thailand, 2024; pp. 11143–11156. <https://doi.org/10.18653/v1/2024.findings-acl.664>.
71. Gao, T.; Yen, H.; Yu, J.; Chen, D. Enabling Large Language Models to Generate Text with Citations. In Proceedings of the Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing; Bouamor, H.; Pino, J.; Bali, K., Eds., Singapore, 2023; pp. 6465–6488. <https://doi.org/10.18653/v1/2023.emnlp-main.398>.
72. Ruhrberg Estévez, S.; Ferber, D.; van der Schaar, M.; Kather, J.N. How to benchmark medical AI agents. *PLOS Medicine* **2026**, *23*, 1–5. <https://doi.org/10.1371/journal.pmed.1005170>.
73. Ma, C.; Zhang, J.; Zhu, Z.; Yang, C.; Yang, Y.; Jin, Y.; Lan, Z.; Kong, L.; He, J. Agentboard: An analytical evaluation board of multi-turn llm agents. *Advances in neural information processing systems* **2024**, *37*, 74325–74362.
74. Mialon, G.; Fourier, C.; Wolf, T.; LeCun, Y.; Scialom, T. GAIA: a benchmark for General AI Assistants. In Proceedings of the The Twelfth International Conference on Learning Representations, 2024.
75. Jimenez, C.E.; Yang, J.; Wettig, A.; Yao, S.; Pei, K.; Press, O.; Narasimhan, K.R. SWE-bench: Can Language Models Resolve Real-world Github Issues? In Proceedings of the The Twelfth International Conference on Learning Representations, 2024.
76. Skarlinski, M.D.; Cox, S.; Laurent, J.M.; Braza, J.D.; Hinks, M.; Hammerling, M.J.; Ponnampati, M.; Rodrigues, S.G.; White, A.D. Language agents achieve superhuman synthesis of scientific knowledge, 2024, [\[arXiv:cs.CL/2409.13740\]](https://arxiv.org/abs/2409.13740).
77. UK National Cyber Security Centre. Prompt injection is not SQL injection (it may be worse), 2025. Accessed 17 July 2026.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.